

ADOPTION

Communications kit

Launch announcements, drip-campaign messages, and FAQ responses for rolling Claude Code out to your engineering organization.

This page is for administrators and engineering leads rolling Claude Code out to a team. It provides copy-ready launch announcements, a tips-and-tricks drip campaign, and one-line FAQ responses for the questions you will be asked most.

ⓘ Treat everything here as draft copy, not finished copy. Rewrite each message in your organization’s voice, swap the example tasks for real bugs and modules from your own codebase, and replace the [bracketed placeholders] before sending. The announcements that drive adoption are the ones that read like someone at your company wrote them.

Launch communications

One announcement in two formats, plus two optional variants. Pick whichever fits your rollout and rewrite it from there.

Before you send

Work through this checklist before the announcement goes out. Each item closes a gap that otherwise turns into a launch-day support thread.

Item	Why it matters
#claude-code channel created and linked in the message	Gives questions one place to land
Install command tested on at least one machine in your environment	Catches proxy or firewall issues before everyone hits them at once
Security and data-handling link ready (<u>Data usage</u> or your internal equivalent)	“Where does my code go?” will be the first reply

Item	Why it matters
One concrete first task chosen, a real bug or file in your codebase	Generic examples don't convert; "fix the flaky test in <code>auth_test.go</code> " does
A named owner for the channel for the first 48 hours	Unanswered launch-day questions kill momentum
A C-suite sponsor lined up to send or co-sign the announcement	Exec-sent launches consistently see higher first-week adoption than admin-sent ones

The announcement

Use this as your standard org-wide rollout message. It covers what Claude Code is, gives a two-minute install path, hands readers one concrete task to try, and answers “where does my code go?” before anyone has to ask.

Email

Subject: Claude Code is live for [Engineering / your team]

Team,

As of today you have access to Claude Code, an AI coding agent that runs in your terminal, reads your actual codebase, and works through real tasks end to end: debugging, refactors, tests, PRs. It is not autocomplete and it is not a chat window. It edits files, runs your commands, and asks permission before anything risky.

Get running in two minutes:

```
curl -fsSL https://claude.ai/install.sh | bash
cd <your-repo>
claude
```

Then run `/init` once. Claude reads your project and writes a `CLAUDE.md` with your build commands and conventions, so you stop re-explaining the basics.

Then try one of these on the repo you are already in:

- "The test in [file] is flaky. Figure out why and fix it"
- "Walk me through how [module] handles [X]"
- "Look at my working diff and tell me what's risky before I push"

Where your code goes: Claude Code runs in your terminal and talks directly to Anthropic's API, with no third-party servers in the loop. It asks before editing files or running commands. Under our Enterprise agreement, Anthropic does not use your code or prompts to train its models. Details: <https://code.claude.com/docs/en/data-usage>
<https://code.claude.com/docs/en/security>

Where to go with questions: `#claude-code`. [Owner name] is watching it

this week.

- [Name]

P.S. Prefer your editor? There is a VS Code extension and a JetBrains plugin. Same agent, no terminal required.

Slack or Teams

 *Claude Code is live for [team]*

AI coding agent, runs in your terminal, reads your repo, does real work:

bugs, refactors, tests, PRs. Asks before it touches anything.

```
`curl -fsSL https://claude.ai/install.sh | bash` → `cd your-repo`  
→ `claude`
```

First thing to try → run ``/init``, then: "the test in [file] is flaky, figure out why and fix it."

 Runs in your terminal, talks only to Anthropic's API. Under our

Enterprise plan your code and prompts are not used to train models.

Data usage → <https://code.claude.com/docs/en/data-usage>

 Quickstart · VS Code · Free 1-hr course

<https://code.claude.com/docs/en/quickstart>

<https://code.claude.com/docs/en/vs-code>

<https://anthropic.skilljar.com/claude-code-in-action>

Questions → this thread. [Owner] is on point.

Subject: One thing I'd like every engineer to try this week

Team,

We have turned on Claude Code for all of engineering. It is an AI agent that works directly in your terminal, on your actual codebase, and the early results from teams already using it are strong enough that I want everyone on it this week.

I am asking for ten minutes:

```
curl -fsSL https://claude.ai/install.sh | bash
cd <your-repo>
claude
```

Then hand it one real task: the bug you have been putting off, or "walk me through how [module] works."

That is the whole ask. [Owner name] and team are in #claude-code for anything you hit along the way.

- [Exec Name]
[Title]

📢 *From [Exec Name]: one thing to try this week*

We have turned on *Claude Code* for all of engineering. Early results are strong enough that I am asking everyone to give it ten minutes on real work this week.

```
`curl -fsSL https://claude.ai/install.sh | bash` → `cd your-repo`  
→  
`claude` → hand it one real task.
```

That's it. Questions → #claude-code.

Executive sponsor variant

Send this from your sponsoring executive, such as the CTO, CIO, or SVP Engineering, under their name and from their account. Launches that go out under an exec's name consistently see higher open rates and faster first-week activation than the same message from an admin or tooling team. It signals a company priority rather than an optional experiment.

This version is deliberately stripped to one ask: install it and run it on one real task. The exec's job is to make the ask land; the standard announcement and `#claude-code` handle the how.

Email

Subject: Claude Code is live for [Engineering / your team]

Team,

As of today you have access to Claude Code, an AI coding agent that runs in your terminal, reads your actual codebase, and works through real tasks end to end: debugging, refactors, tests, PRs. It is not autocomplete and it is not a chat window. It edits files, runs your commands, and asks permission before anything risky.

Get running in two minutes:

```
curl -fsSL https://claude.ai/install.sh | bash
cd <your-repo>
claude
```

Then run `/init` once. Claude reads your project and writes a `CLAUDE.md` with your build commands and conventions, so you stop re-explaining the basics.

Then try one of these on the repo you are already in:

- "The test in [file] is flaky. Figure out why and fix it"
- "Walk me through how [module] handles [X]"
- "Look at my working diff and tell me what's risky before I push"

Where your code goes: Claude Code runs in your terminal and talks directly to Anthropic's API, with no third-party servers in the loop. It asks before editing files or running commands. Under our Enterprise agreement, Anthropic does not use your code or prompts to train its models. Details: <https://code.claude.com/docs/en/data-usage>
<https://code.claude.com/docs/en/security>

Where to go with questions: `#claude-code`. [Owner name] is watching it

this week.

- [Name]

P.S. Prefer your editor? There is a VS Code extension and a JetBrains plugin. Same agent, no terminal required.

Slack or Teams

 *Claude Code is live for [team]*

AI coding agent, runs in your terminal, reads your repo, does real work:

bugs, refactors, tests, PRs. Asks before it touches anything.

```
`curl -fsSL https://claude.ai/install.sh | bash` → `cd your-repo`  
→ `claude`
```

First thing to try → run ``/init``, then: "the test in [file] is flaky, figure out why and fix it."

 Runs in your terminal, talks only to Anthropic's API. Under our

Enterprise plan your code and prompts are not used to train models.

Data usage → <https://code.claude.com/docs/en/data-usage>

 Quickstart · VS Code · Free 1-hr course

<https://code.claude.com/docs/en/quickstart>

<https://code.claude.com/docs/en/vs-code>

<https://anthropic.skilljar.com/claude-code-in-action>

Questions → this thread. [Owner] is on point.

Subject: One thing I'd like every engineer to try this week

Team,

We have turned on Claude Code for all of engineering. It is an AI agent that works directly in your terminal, on your actual codebase, and the early results from teams already using it are strong enough that I want everyone on it this week.

I am asking for ten minutes:

```
curl -fsSL https://claude.ai/install.sh | bash
cd <your-repo>
claude
```

Then hand it one real task: the bug you have been putting off, or "walk me through how [module] works."

That is the whole ask. [Owner name] and team are in #claude-code for anything you hit along the way.

- [Exec Name]
[Title]

 *From [Exec Name]: one thing to try this week*

We have turned on *Claude Code* for all of engineering. Early results are strong enough that I am asking everyone to give it ten minutes on real work this week.

```
`curl -fsSL https://claude.ai/install.sh | bash` → `cd your-repo`  
→  
`claude` → hand it one real task.
```

That's it. Questions → #claude-code.

Pilot group variant

Use for a phased rollout. Send to the pilot cohort only.

Subject: You're in the Claude Code pilot

[Name / team],

You are in the first wave of Claude Code at [company]. We picked this group because you will put it on real problems and tell us the truth about it.

The ask: use it on at least one real task this week, then drop a note in #claude-code-pilot covering what worked, what was annoying, and what surprised you. That feedback decides how we roll it out to everyone else.

[Continue with "Get running in two minutes" from the standard announcement]

One extra thing for pilots: on your first multi-file change, press Shift+Tab until you see "plan". Claude will lay out exactly what it intends to do before it touches a file. It is the fastest way to calibrate how much to trust it.

Champion recruitment DM

After launch, DM the two or three people who are most active in #claude-code .

Hey [name], your #claude-code posts are doing more for adoption than my announcement did. A couple of people told me your [thread / screenshot] was why they actually tried it.

Want to make that semi-official? Low lift: mostly keep posting what you are posting, plus first crack at new features and a direct line to the Anthropic team. I can share a short playbook if you're in.

Tips and tricks campaign

Ready-to-paste Slack or Teams messages designed to drive feature activation after launch. Each follows the same pattern: a hook, the payoff, a “try it now” prompt, and a docs link. Drip them one or two a week in `#claude-code`, or pick the handful that match your team’s gaps. They stand alone with no required order.

Copy the message body from each block directly into Slack or Teams. Replace `[bracketed placeholders]` before sending.

Get started

Choosing the right model

 ***Tip: Match the model to the moment***

Using Opus to fix a typo burns compute. Using Haiku for a 12-file refactor is asking for a re-do.

Claude Code runs on the same models as the Claude app, and you can switch mid-session. ***Sonnet*** is the workhorse default for everyday feature work, bugs, tests, and reviews. Reach for ***Opus*** on large refactors, gnarly debugging, or anything high-stakes. Drop to ***Haiku*** for quick questions, formatting, and mechanical edits where speed wins. ***Fable 5*** is the most capable model for your hardest, longest-running tasks; it is not the default, so select it with ``/model fable``, and note that cybersecurity and biology content falls back to Opus automatically.

Try it now: type ``/model`` and pick Sonnet if you haven't already. It is the right default for most tasks.

 Model configuration → <https://code.claude.com/docs/en/model-config>

Model	Best for
Fable 5	The hardest, longest-running tasks. Opt-in only: select it with <code>/model fable</code> . Cybersecurity or biology content <u>falls back to Opus</u>
Opus	Large-scale refactors, complex debugging, architecture decisions, high-stakes changes
Sonnet	Everyday feature work, bug fixes, tests, documentation, code review. Recommended default.
Haiku	Quick questions, formatting, mechanical edits, rapid iteration

Quick wins to try first



Tip: Three things to try in your first 10 minutes

Installed Claude Code but not sure what to actually ask it? Start with the stuff that has been bugging you all week.

- Fix something annoying: "the test in `[file]` is flaky, figure out why"
- Get oriented in code you didn't write: "walk me through how `[module]` works"
- Sanity-check before you push: "look at my working diff and tell me what looks risky"

None of these need setup. Just ``cd`` into your repo and run ``claude``.

Try it now: pick the bug you have been avoiding and paste the error message in.

 Quickstart → <https://code.claude.com/docs/en/quickstart>

Project memory

`/init` and **CLAUDE.md**

📁 *Tip: Stop re-explaining your repo every session*

Telling Claude "we use pnpm, not npm" for the fifth time? There is a one-time fix.

Run `"/init"` once per repo. Claude reads your project structure and writes a CLAUDE.md file with your build commands, architecture, and conventions.

Every future session in that repo starts from this file automatically. Keep it under two screens. It is a cheat sheet, not documentation.

Try it now: open your main repo, run `'claude'`, type `"/init"`. Thirty seconds, pays off every session after.

📖 CLAUDE.md and project memory → <https://code.claude.com/docs/en/memory>

@-references

📌 *Tip: Stop pasting file contents into the chat*

Copying 200 lines of a component into your prompt so Claude can "see" it? You don't have to.

Type `'@'` then a file path. Claude pulls the file directly into context. Works for whole directories too.

```
> the styles in @src/components/Button.tsx look off, check  
against  
> @docs/design-system.md
```

Try it now: type `'@'` then Tab. Autocomplete shows you every file in reach.

📖 Referencing files → <https://code.claude.com/docs/en/common-workflows>

Control and safety

Permission modes

🛡️ ***Tip:** One keystroke between "look but don't touch" and "just do it"

Sometimes you want Claude to ask before every edit. Sometimes you just want it to ship. You shouldn't have to pick one forever.

Shift+Tab cycles through how much leash Claude gets: ***default*** asks before risky stuff, ***acceptEdits*** lets file edits and common filesystem commands flow through while still checking before other shell commands, and ***plan*** proposes changes for your approval before anything is touched. Plan mode is the trust-builder, so start there for anything touching multiple files.

Try it now: on your next refactor, hit Shift+Tab until you see "plan", then describe the change. You'll get a full proposal before a single file moves.

📖 Permission modes → <https://code.claude.com/docs/en/permissions>

Checkpointing and `/rewind`

⏮ *Tip: There is an undo button for the whole conversation*

Claude went down the wrong path three turns ago and now you're untangling it? You don't have to fix forward.

`/rewind` rolls back to an earlier point in the conversation, including the file changes Claude made along the way. Checkpointing is automatic; you don't set anything up.

Try it now: press `*Esc*` twice to open the rewind menu, or type `/rewind`.
Pick the point before things went sideways.

📖 Checkpointing → <https://code.claude.com/docs/en/checkpointing>

Connect your tools

MCP connectors

🔗 *Tip: Let Claude read your issue tracker so you don't have to paste tickets*

Copy-pasting Jira tickets into the terminal feels like a step backward.
It is.

One config file (`.mcp.json` at your project root) wires Claude into GitHub, Jira, Linear, or whatever tracker you use. Then "what's the top-priority issue assigned to me?" and "go ahead and fix it" happen in the same conversation.

Try it now: ask Claude "set up an MCP connector for [GitHub/Jira/Linear] in this repo". It will write the config for you.

📖 MCP connectors → <https://code.claude.com/docs/en/mcp>

Automate your workflows

Skills

⚡ *Tip: Turn that prompt you keep retyping into a command*

Typed "summarize what I worked on today from git log, format it for standup"
three times this week? That's a slash command waiting to happen.

A SKILL.md file in ``.claude/skills/<name>/`` becomes a reusable prompt; type ``/name`` to run it. Make one the second time you type a multi-step prompt you've typed before. Easiest path: ask Claude to make it for you.

Try it now: type "make me a /standup skill that summarizes what I worked on today from git log", then run ``/standup`` tomorrow morning.

📖 Skills → <https://code.claude.com/docs/en/skills>

Hooks

🔔 *Tip: Get pinged when your refactor finishes*

Sitting at your desk watching Claude work through a long task?
You've got better things to do for those eight minutes.

Hooks are shell commands that fire on Claude Code events. A Stop hook that sends a desktop notification means you can kick off a long refactor, walk away, and get pinged the moment it's done.

Try it now: ask Claude "add a Stop hook that sends a desktop notification when you finish". It will write the script and wire it up.

📖 Hooks guide → <https://code.claude.com/docs/en/hooks-guide>

Day-to-day development

Screenshots and images

 *Tip: Stop describing the error dialog. Just show it.*

Typing out "there's a red box that says something about a null reference and it's pointing at line 47-ish"? Screenshot it.

Drag a screenshot straight into the terminal and Claude sees it: error dialogs, UI mockups, whiteboard photos, Figma exports. *Ctrl+V* pastes from clipboard (use Ctrl+V on macOS too, not Cmd+V).

Try it now: next time something visual breaks, screenshot it and paste it right into the prompt. Then just type "what's wrong here?"

 Working with images → <https://code.claude.com/docs/en/common-workflows>

Git workflows

🌿 *Tip: Hand off the whole git ceremony*

The fix took 5 minutes. The commit message, branch, and PR description took 15. That ratio is wrong.

Claude handles the full git flow: commits with conventional messages, branches, PRs with proper summaries. One ask: "fix the off-by-one, commit with a conventional commit message, and open a PR." Reviewing someone else's work? Paste the PR URL and ask Claude to walk you through the diff.

Try it now: after your next fix, instead of switching to your git client, just type "commit this with a good message and open a PR".

📖 Creating pull requests → <https://code.claude.com/docs/en/common-workflows>

Share and scale

Plugins

📦 *Tip: Someone probably already built that skill*

About to spend an hour building a `/deploy` command? Check if it already exists.

Skills get bundled and shared as plugins. `/plugin` browses what's available and installs in one step. Five minutes of browsing can save an hour of building.

Try it now: type `/plugin` and scroll through. You'll find at least one thing you didn't know you wanted.

📖 Plugins → <https://code.claude.com/docs/en/plugins>

Security and admin

Security architecture

 *Tip: The answer to "is this safe?" for the next time you're asked*

Someone on your team is going to ask "wait, where does my code go?"

Here's the short version you can paste.

Permission-first by design. Every file edit, shell command, and external call is gated by your approval. The CLI runs in your terminal and talks directly to Anthropic's API, with no third-party servers, and supports optional OS-level sandboxing for shell commands. Under our Enterprise plan, Anthropic does not use your code or prompts to train its models.

Try it now: save these two links for the next time the question comes up. They answer most security-review questions.

-  <https://code.claude.com/docs/en/security>
-  <https://code.claude.com/docs/en/data-usage>

Best practices

✅ *Tip: The 4 habits that separate "tried it once" from "use it daily"*

Most people who bounce off Claude Code skipped one of these. Most people who stick did all four in week one.

- Start in plan mode for anything touching multiple files
- Run `/init` early; context compounds
- Review diffs before committing; Claude can be confidently wrong
- Verify changes that touch critical paths; treat it like a sharp junior, not an oracle

Try it now: if you've only done one or two of these, pick the one you're missing and do it on your next task. Post what changed in `#claude-code`.

📄 Best practices → <https://code.claude.com/docs/en/best-practices>

Quick reference

FAQ responses

One-line replies for the questions you will be asked most.

Question	Response
"Does it work in VS Code?"	Yes. There is a VS Code extension and a JetBrains plugin with the same features, embedded in your editor. VS Code →
"Do I have to configure anything first?"	No. Install, then run <code>claude</code> in any repo. Run <code>/init</code> once and you're set. Quickstart →
"Where does my code go?"	The CLI runs in your terminal and sends context to Anthropic's API for inference, with no third-party servers. Under your Enterprise plan, your code and prompts are not used to train models. Data usage →
"Can it see my whole repo?"	It reads what you give it access to. File reads inside your working directory don't prompt; permission prompts gate edits, shell commands, and anything outside that directory. Permissions →
"How is this different from	Copilot autocompletes lines. Claude Code is an agent that reads files, runs

Question	Response
Copilot?"	commands, and makes multi-file edits. Overview →
"What should I try first?"	A bug you've been putting off because it's tedious. "The test in [file] is flaky, figure out why." Quickstart →

Prompt templates

Share these starter prompts with engineers who have installed but aren't sure what to ask. Each one is phrased the way it would be typed into a real session; replace the bracketed pieces with files from your own repo.

Task	Prompt
Fix a bug	"the tests in [file] are failing, figure out why and fix it"
Understand code	"walk me through how [module] works, then tell me where the entry point is"
Safe refactor	"refactor [module] to [goal], use plan mode so I can review first"
Write tests	"write tests for [file] that cover the edge cases around [scenario]"
Review before commit	"look at my working diff and tell me what looks risky"
Open a PR	"fix [issue], write a conventional commit, and open a PR with a summary"
Make a skill	"make me a /ship skill that runs tests and lint before commit"
Debug a stack trace	"here's the stack trace, find the root cause, don't just paper over it"

💡 Claude Code ships frequently. Verify version-specific details against the [documentation home page](#) before distributing internally.